```
=== RUN: Command Test: dump cstest version
--- PASS
duration: 1.819290143s
stdout: 1.19.3

=== RUN: File Existence Test: downloaded docker.tgz absence in /tmp
--- FAIL
duration: 0s
Error: File /tmp/docker.tgz should not exist but does
=== RUN: File Existence Test: extracted docker folder absence in /tmp
--- FAIL
duration: 0s
Error: File /tmp/docker should not exist but does
=== RUN: File Existence Test: curl absence in /usr/bin
--- FAIL
duration: 0s
Error: File /usr/bin/curl should not exist but does
=== RUN: File Existence Test: ca-certificates absence in /etc
--- FAIL
duration: 0s
Error: File /etc/ca-certificates should not exist but does
=== RUN: File Existence Test: cstest absence in /sbin
--- PASS
duration: 0s
=== RUN: File Existence Test: cstest absence in /bin
--- PASS
duration: 0s
=== RUN: Metadata Test
--- PASS
duration: 0s

----------------------------------
------------- RESULTS ------------
----------------------------------
Passes:      4
Failures:    4
```

Figure 1: Output showing the CST tests that have failed

```
=== RUN: Command Test: dump cstest version
--- PASS
duration: 2.086098983s
stdout: 1.19.3

=== RUN: File Existence Test: downloaded docker.tgz absence in /tmp
--- PASS
duration: 0s
=== RUN: File Existence Test: extracted docker folder absence in /tmp
--- PASS
duration: 0s
=== RUN: File Existence Test: curl absence in /usr/bin
--- PASS
duration: 0s
=== RUN: File Existence Test: ca-certificates absence in /etc
--- PASS
duration: 0s
=== RUN: File Existence Test: cstest absence in /sbin
--- PASS
duration: 0s
=== RUN: File Existence Test: cstest absence in /bin
--- PASS
duration: 0s
=== RUN: Metadata Test
--- PASS
duration: 0s

----------------------------------
------------- RESULTS ------------
----------------------------------
Passes:      8
Failures:    0
Duration:    2.086098983s
Total tests: 8

PASS
```

Figure 2: CST tests that have passed

commands run properly in the target image. Regexes can be used to check for expected or excluded strings in both *stdout* and *stderr*. The *fileExistenceTests* section ensures that desired files and/or directories are present and/or absent in a container image. The Metadata test ensures the container is configured correctly. The CST also provides file content tests to open a file in the container image system and check its contents. All these checks are optional and you can choose your tests based on your needs. All the tests provided by CST have their property fields to finetune the tests. Please go through the CST GitHub documentation to explore these test fields.

Let's fix the structure of our problematic Docker image by creating *Dockerfile_CSTestFxd* as per the diff shown below:

```
21c21,24
<    && tar zxvf docker.tgz
---
>    && tar zxvf docker.tgz \
>    && mv docker/docker /usr/local/bin \
>    && rm -rfv docker docker.tgz \
>    && apk del .dwnld-deps
```

Execute the command *docker build --no-cache --progress plain . -f Dockerfile_CSTestFxd -t cstestfxd:1.19.3* in your terminal to build a new, structurally corrected Docker image.

The command *docker run -it --rm -v ${PWD}:/etc/cstest:ro -v /var/run/docker.sock:/var/run/docker.sock:ro ghcr.io/googlecontainertools/container-structure-test:1.19.3 test -c /etc/cstest/config.yaml -i cstestfxd:1.19.3* triggers another static test run to verify the structure of the corrected image. Voila! All the structural tests are passing now against the corrected image, as shown in Figure 2.

Now you have enough working knowledge to make CST part of your acceptance testing pipeline. The CST GitHub README is a great place to explore this amazing framework.

Runtime acceptance testing for containers with dgoss

GOlang based **S**erver **S**pec is a lightweight, no dependencies, FOSS solution you can bake into your servers to achieve the much-needed machine level acceptance testing. It's a *golang* static binary (macOS and Windows binaries are in alpha, currently). So put it in your servers, define your acceptance tests in their respective YAML files, feed those to the running goss, and that's it! The goss configuration is built around a set of resources and their properties creating a set of test criteria. The goss run checks the current state of those resources in your running machine and declares them passed (or green) or failed (or red), accordingly. The default test configuration file is *goss.yaml* but that could be changed using the *-g* global option, as we have done in our case. The goss requires an action argument which is *v* (or *validate*) in our case. Typing only *goss* on the command line dumps all the possible actions as well as various global options. Every goss action can take a set of options, which is dumped when you suffix that with the *-h* (or *--help*) option. The *-f* (or *--format*) option to validate dumps the test report in various formats including *documentation*, *json*, *junit*, *nagios*, etc. There is also a *silent* format which doesn't dump anything but only indicates overall tests passed or failed when accessed through the